

FEED — SYSTEM DOCUMENTATION

FeedWorld & EPM Architecture Guide

Plain-English UML diagrams and database structure for the two features you asked about: the **FeedWorld publications page** and the **EPM (Export Promotional Meetings) page** — covering both the React frontend (`C:\Users\Pavan\feedorg`) and the Spring Boot backend (`C:\Users\Pavan\feed backend\feed`).

FeedWorld / Publications

No database at all — every publication is a folder on disk. Simple by design.

EPM Events, Registration & Volunteers

Backed by 3 real MySQL tables via Spring Data JPA.

EPM Gallery

Same folder-based design as FeedWorld — one folder per photo section, no database.

Contents

1. FeedWorld (Publications) — What it does
2. FeedWorld — Frontend structure
3. FeedWorld — Backend architecture
4. FeedWorld — Backend class diagram
5. FeedWorld — Storage layout (no database)
6. EPM — What it does
7. EPM — Frontend structure (10 pages)
8. EPM — Backend architecture (two designs in one module)
9. EPM — Backend class diagram: Events, Registrations, Volunteers
10. EPM — Backend class diagram: Gallery
11. EPM — Database ER diagram
12. EPM — Gallery storage layout (no database)
13. Database table reference (exact columns, from the live database)

How to read these: boxes are things that exist in the code (a page, a class, a table); arrows show "talks to" or "calls"; a cylinder shape is somewhere data is actually stored (a database table, or a folder of files).

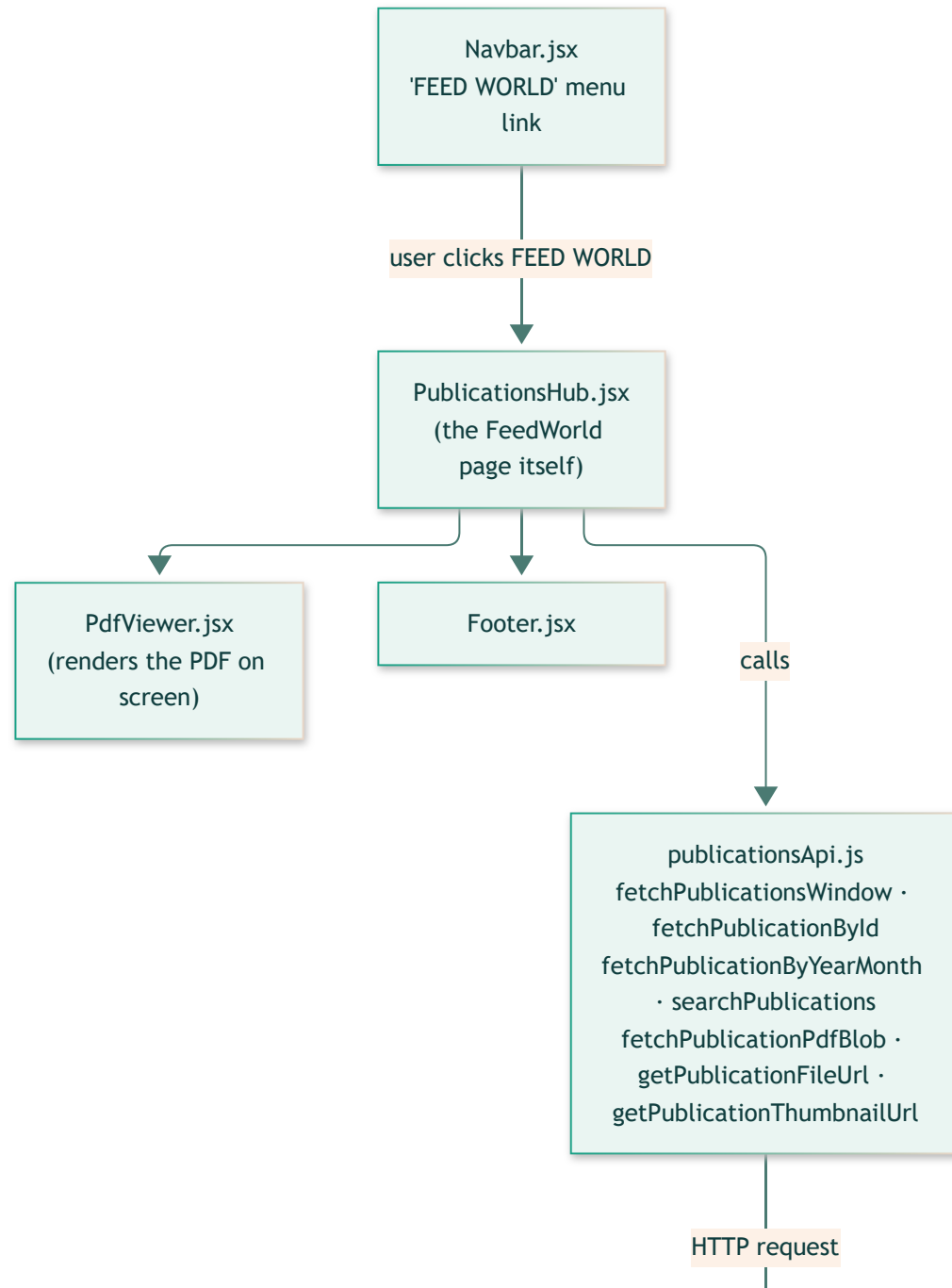
1. FeedWorld (Publications) — What it does

The FeedWorld page is where a logged-in user picks a **year and month** and reads that issue's PDF magazine. They can also search, or step through an archive of past issues.

The one rule that shapes this whole module: the PDF is never pulled from a database. Every publication is just a folder on the server's hard disk — `storage/publications/<year>/<month>/` — holding the PDF file itself, a small cover-thumbnail image, and a `meta.json` text file with the title, volume number, etc. When the backend needs to answer "what publications exist" or "give me the PDF for August 2025", it simply looks at what folders and files are sitting on disk right now. There is no publications table in MySQL — we confirmed this directly against the live database (`SHOW TABLES` on `feed_db` lists no such table).

2. FeedWorld — Frontend structure

One page component does all the work. `PublicationsHub.jsx` is what the user sees at the "FEED WORLD" menu link: a year/month finder, a search box, an archive list on the left, and a PDF viewer on the right. It only fetches data when the user actually clicks something — nothing loads automatically in the background.



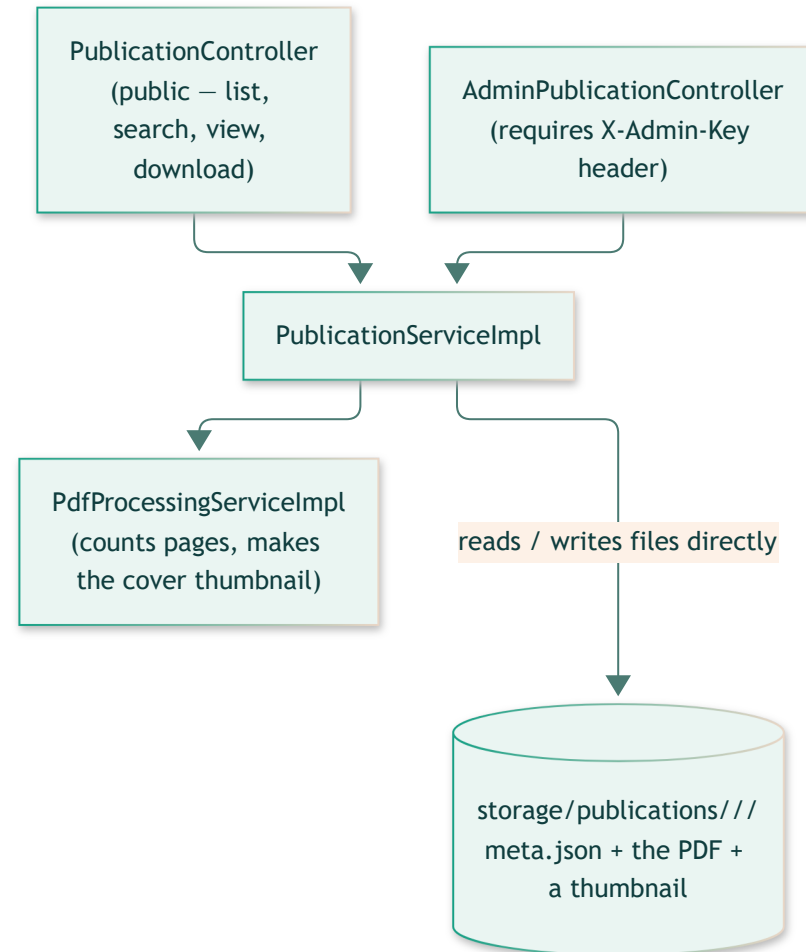


Backend REST API
/api/publications/...

Everything the user does on this page funnels through one API helper file, which talks to the backend over plain HTTP.

3. FeedWorld — Backend architecture

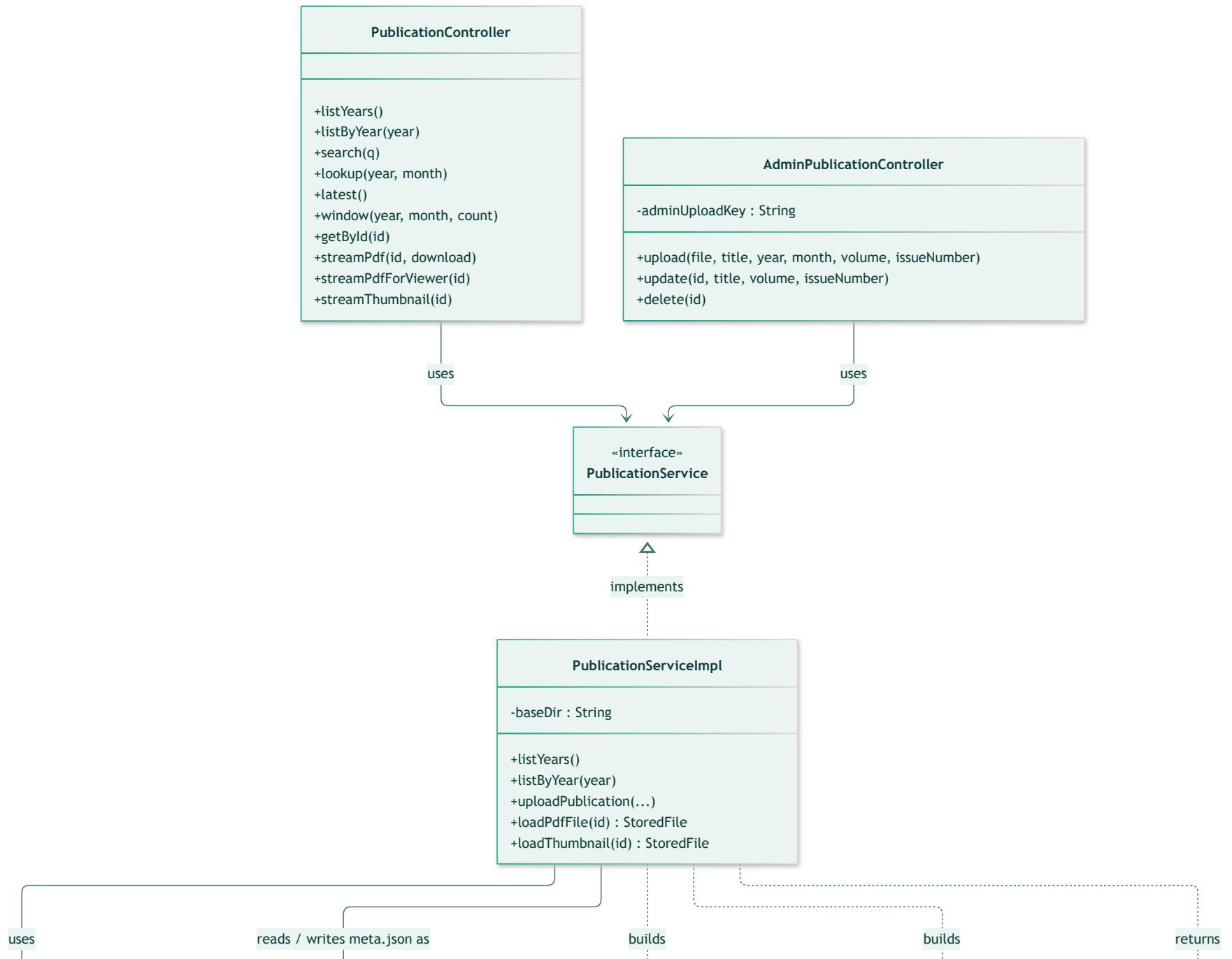
Two controllers guard the same publication folders: one is public and read-only (anyone logged in can browse and view), the other requires a secret admin key and is the only way to upload, edit, or delete an issue. Both hand off to one service class that does the real work of reading and writing files. Notice there is no database box in this picture at all — the service talks straight to the disk.

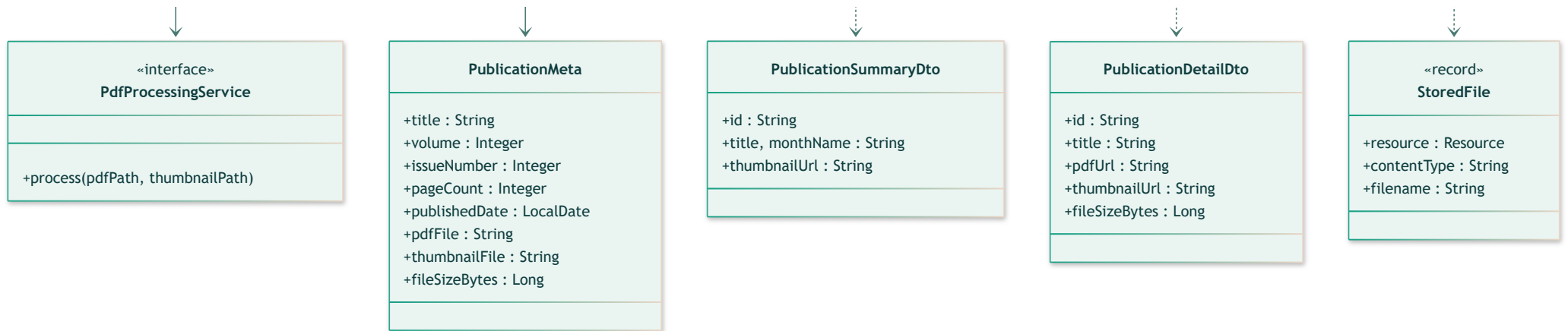


No database involved: this whole feature never runs a single SQL query. "Storage" here means files on the server's disk, organized by folder name — not a database table.

4. FeedWorld — Backend class diagram

This is the same picture as above, but at code level — the actual Java classes, their key methods, and what each one holds.

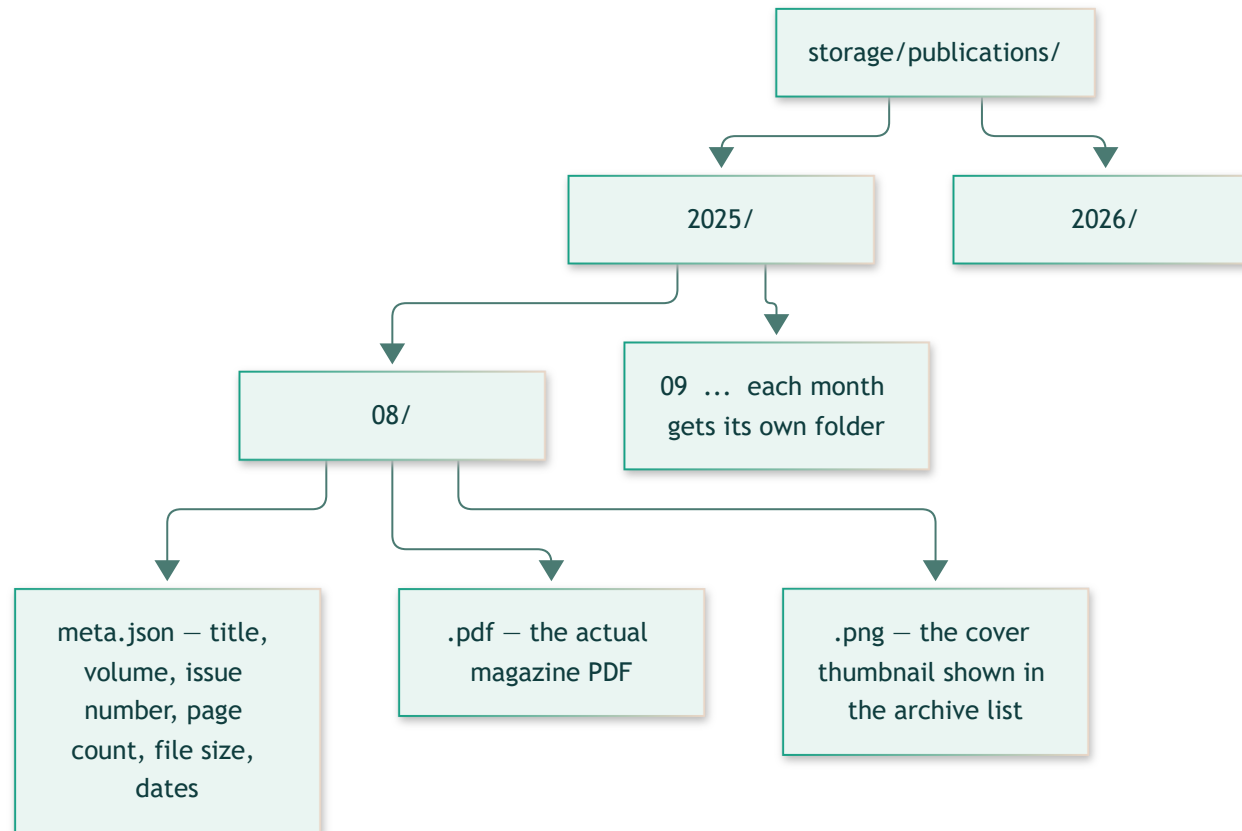




Dashed arrows mean "produces/uses this data shape"; solid arrows mean "calls this class directly".

5. FeedWorld — Storage layout (no database)

This is what actually sits on the server's disk. The year and month are the folder names themselves — that's how the app knows "2025 / August" exists, with no database row required.



6. EPM — What it does

EPM stands for **Export Promotional Meetings** — real-world events FEED organizes across India. The EPM section of the site is actually **10 separate pages** built around one shared idea: list events, let people register or volunteer for one, and show photos from past meetings.

Structurally, EPM is a hybrid of the two storage philosophies in this codebase:

Events, Registrations & Volunteers

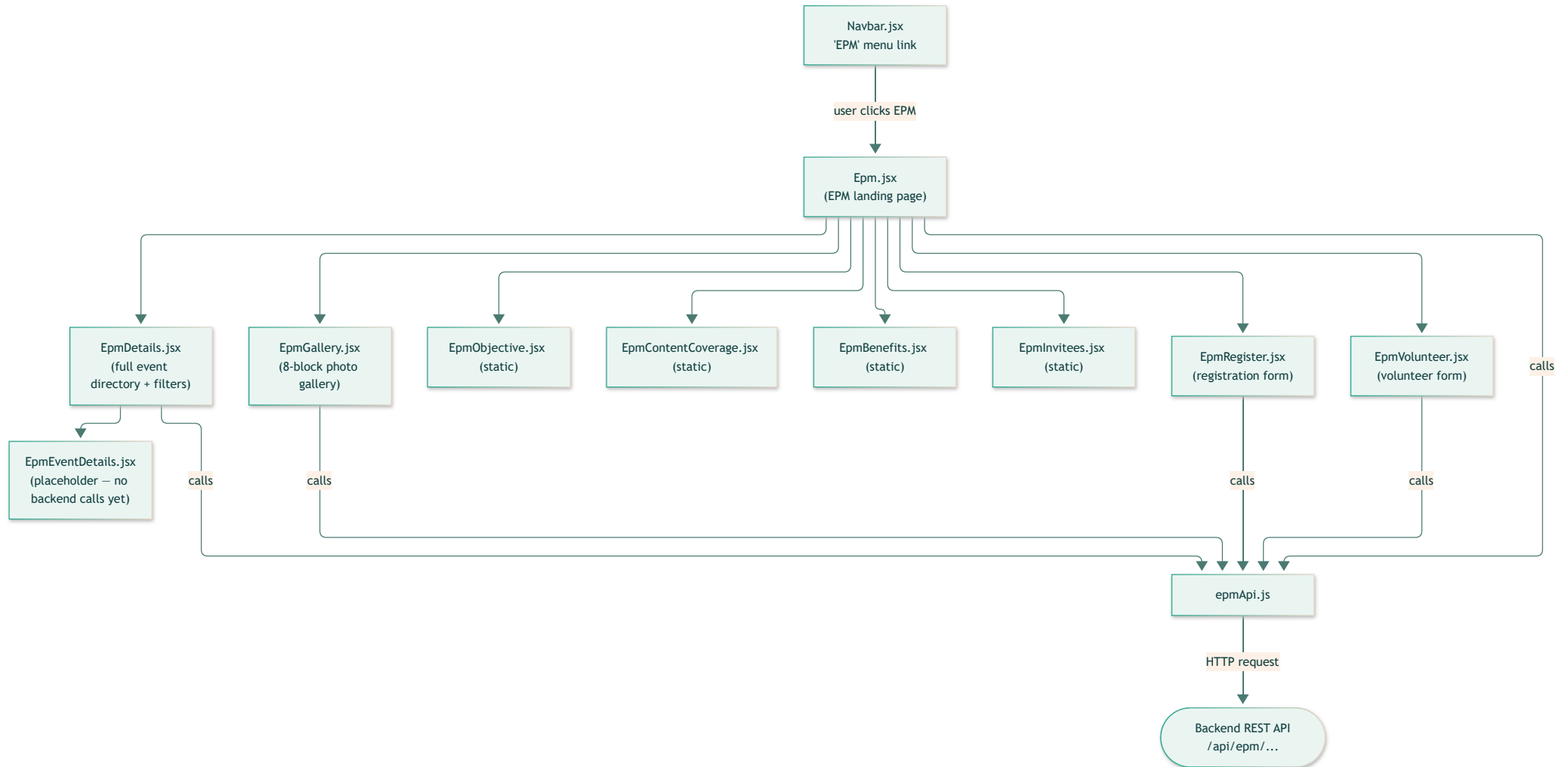
These are real rows in a real MySQL database (via Spring Data JPA) — because an event needs to be looked up, filtered, and counted in ways a folder of files can't easily do.

The photo Gallery

This deliberately follows the exact same no-database, folder-per-section design as FeedWorld — see sections 10 & 12.

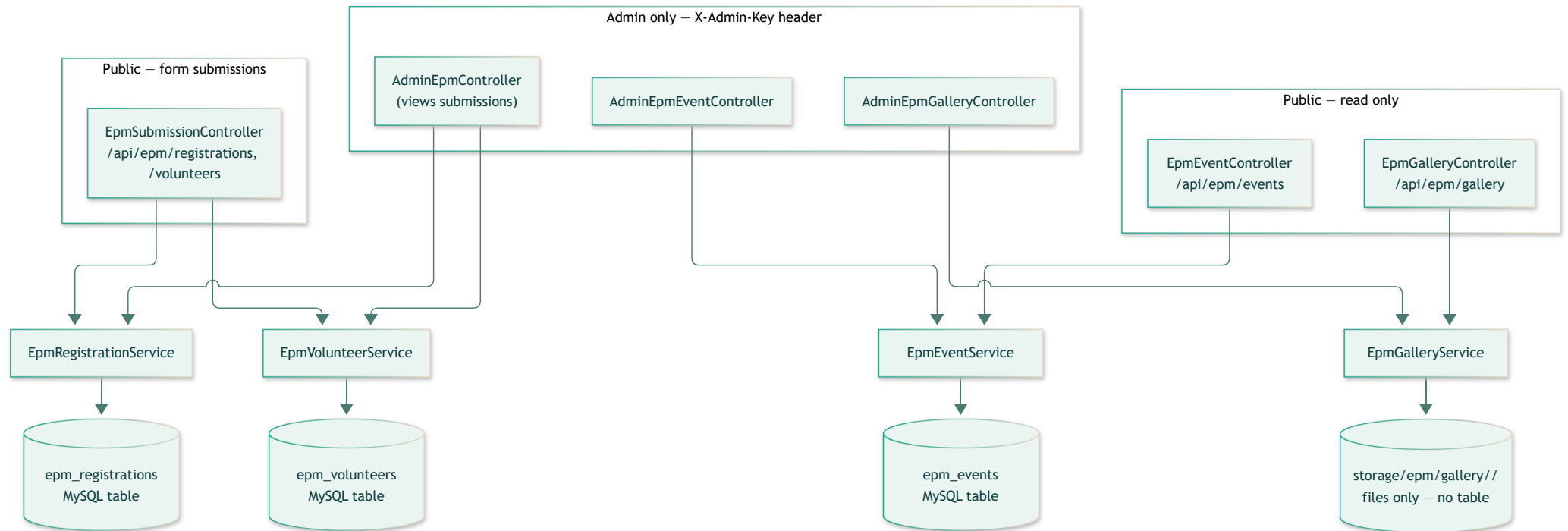
7. EPM — Frontend structure (10 pages)

`Epm.jsx` is the landing page (hero, stats, upcoming events, a gallery teaser, testimonials) and is the door into everything else. Four of the pages below are plain static "read more" pages with no backend calls at all. `EpmEventDetails.jsx` is currently just a placeholder screen — it doesn't call the backend yet, even though an endpoint for it already exists (`fetchEpmEventById`).



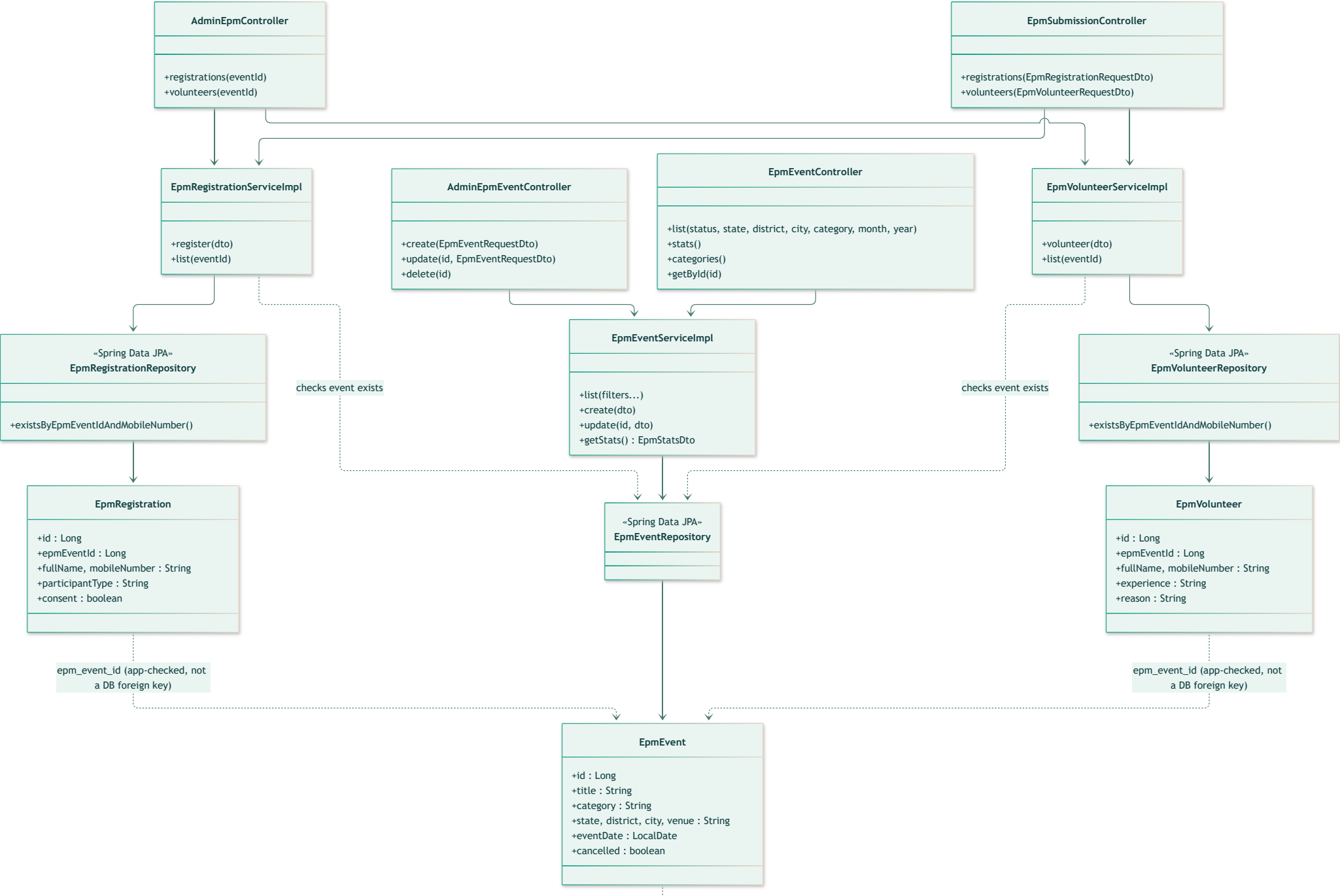
8. EPM — Backend architecture (two designs in one module)

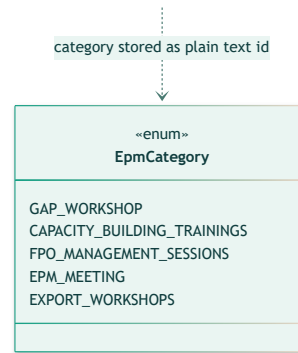
This is the key picture for understanding EPM: the left half (Events / Registrations / Volunteers) goes through Spring Data JPA into real MySQL tables. The right half (Gallery) never touches the database at all — same folder-per-section design as FeedWorld.



9. EPM — Backend class diagram: Events, Registrations & Volunteers

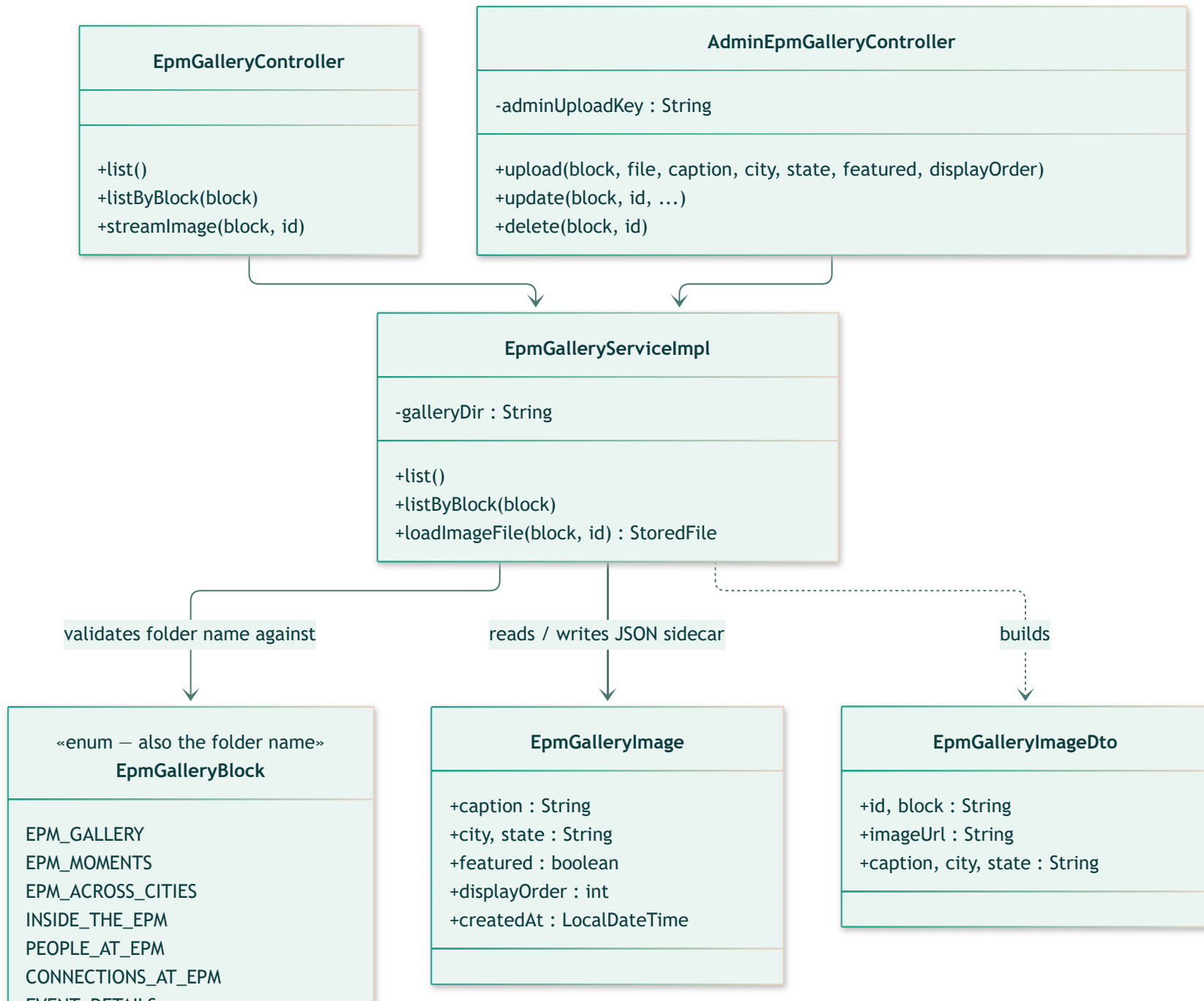
A registration or volunteer sign-up always points at one event (by its id), and snapshots that event's city/state/date at the moment of signing up — so even if the event details change later, the registration still shows what it looked like when the person signed up. Note the dotted line: the app checks the event exists before saving a registration, but there is no formal foreign-key constraint enforcing it at the database level (explained more in section 11).





10. EPM — Backend class diagram: Gallery

This mirrors the FeedWorld design exactly (section 4), just with "block" (a gallery section like "EPM Moments") standing in for "year/month". `EpmGalleryBlock` is the fixed list of 8 folders the gallery page is allowed to read from.



EVENT_DETAILS
THE_EPM_EXPERIENCE

11. EPM — Database ER diagram

Three tables, all confirmed directly against the live `feed_db` database. The lines below show how they relate *logically* — an event has many registrations and many volunteers — but read the fine print: MySQL itself has no foreign-key constraint wiring these together (Hibernate never generated one, because the Java code uses a plain number field rather than a formal JPA relationship). The app enforces the link in code instead, by looking the event up before saving.

epm_events		
bigint	id	PK
varchar	title	
varchar	category	
varchar	state	
varchar	district	
varchar	city	
varchar	venue	
date	event_date	
varchar	time_range	
bit	cancelled	
datetime	created_at	
datetime	updated_at	

one event, many signups (app-enforced)

one event, many signups (app-enforced)

epm_registrations			
bigint	id	PK	

epm_volunteers			
bigint	id	PK	

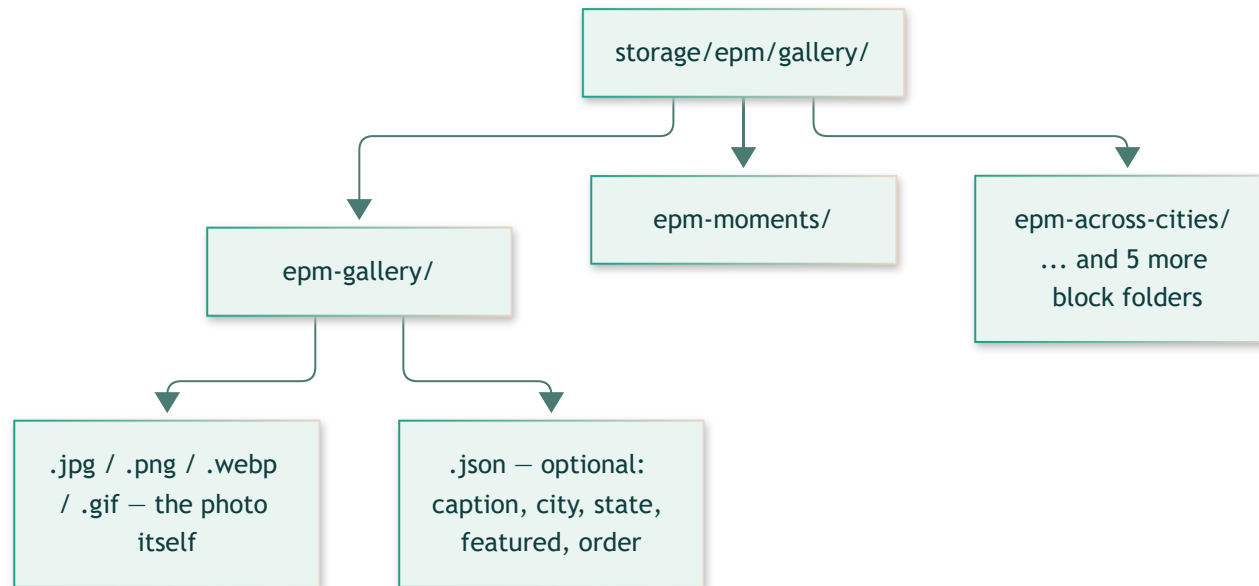
datatype	column name	is primary key	is foreign key
bigint	epm_event_id		no DB foreign key
varchar	event_city		
varchar	event_state		
date	event_date		
varchar	full_name		
varchar	mobile_number		
varchar	email		
varchar	state		
varchar	district		
varchar	participant_type		
bit	consent		
datetime	created_at		

datatype	column name	is primary key	is foreign key
bigint	epm_event_id		no DB foreign key
varchar	event_city		
varchar	event_state		
date	event_date		
varchar	full_name		
varchar	mobile_number		
varchar	email		
varchar	state		
varchar	district		
varchar	experience		
text	reason		
datetime	created_at		

Also worth knowing: registering or volunteering needs no login — neither table has any link to the `users` table. Someone just types their name/phone/email into the form each time.

12. EPM — Gallery storage layout (no database)

Exactly the same idea as FeedWorld's publication folders (section 5) — one folder per gallery section, and dropping an image file straight into the right folder is enough for it to show up on the page. No upload form, no database row required.



13. Database table reference (live schema)

These are the exact columns as they exist right now in the MySQL `feed_db` database — pulled directly from `DESCRIBE` / `SHOW CREATE TABLE`, not guessed from code. FeedWorld/Publications and the EPM Gallery have **no tables at all** — they are intentionally left out of this list because they genuinely don't exist.

epm_events

Column	Type	Nullable	Notes
id	bigint	No — Primary Key	Auto-increment
title	varchar(255)	No	
category	varchar(255)	Yes	Free text, checked in code against the 5 fixed category names
state / district / city / venue	varchar(255)	No	Where the event happens
event_date	date	No	
time_range	varchar(255)	Yes	Free text, e.g. "10:00 AM – 02:00 PM"
cancelled	bit(1)	No	Default false
created_at / updated_at	datetime(6)	created_at: No, updated_at: Yes	

epm_registrations

Column	Type	Nullable	Notes
id	bigint	No — Primary Key	Auto-increment
epm_event_id	bigint	No	Points at <code>epm_events.id</code> , but only in application code — no DB-level foreign key
event_city / event_state / event_date	varchar / varchar / date	No	Snapshot of the event at signup time
full_name / mobile_number	varchar(255)	No	Mobile must be unique per event (checked in code, not a DB constraint)
email	varchar(255)	Yes	Optional
state / district	varchar(255)	No	The registrant's own state/district (not the event's)
participant_type	varchar(255)	No	

consent	bit(1)	No
created_at	datetime(6)	No

epm_volunteers

Column	Type	Nullable	Notes
id	bigint	No — Primary Key	Auto-increment
epm_event_id	bigint	No	Same app-only reference as epm_registrations
event_city / event_state / event_date	varchar / varchar / date	No	Snapshot of the event at signup time
full_name / mobile_number	varchar(255)	No	
email	varchar(255)	Yes	Optional
state / district	varchar(255)	No	
experience	varchar(255)	No	e.g. their relevant background
reason	text	Yes	Why they want to volunteer — free text, optional
created_at	datetime(6)	No	

For reference, two more tables exist in the same database but belong to the shared login system, not to either page covered here: `users` (has a `profile_image_path` column — same disk-file-plus-path-only pattern) and `user_sessions`.